OneLive: Dynamically Unified Generative Framework for Live-Streaming Recommendation

Shen Wang[†], Yusheng Huang[†], Ruochen Yang[†*], Shuang Wen[†], Pengbo Xu[†], Jiangxia Cao[‡], Yueyang Liu, Kuo Cai, Chengcheng Guo, Shiyao Wang, Xinchen Luo, Qiang Luo, Ruiming Tang, Shuang Yang, Zhaojie Liu, Guorui Zhou, Han Li, Kun Gai
Kuaishou Technology, Beijing, China
{wangshen, huangyusheng, yangruochen, wenshuang, xupengbo, caojiangxia, liuyueyang05, caikuo, guochengcheng03, wangshiyao08, luoxinchen, luoqiang, tangruiming, yangshuang08, zhaotianxing, zhouguorui, lihan08}@kuaishou.com, kun.gai@qq.com

Abstract

Live-streaming recommender system serves as critical infrastructure that bridges the patterns of real-time interactions between users and authors. Similar to traditional industrial recommender systems, live-streaming recommendation also relies on cascade architectures to support large-scale concurrency. Recent advances in generative recommendation unify the multi-stage recommendation process with Transformer-based architectures, offering improved scalability and higher computational efficiency. However, the inherent complexity of live-streaming prevents the direct transfer of these methods to live-streaming scenario, where continuously evolving content, limited lifecycles, strict real-time constraints, and heterogeneous multi-objectives introduce unique challenges that invalidate static tokenization and conventional model framework.

To address these issues, we propose **OneLive**, a dynamically unified generative recommendation framework tailored for live-streaming scenario. OneLive integrates four key components: (i) A **Dynamic Tokenizer** that continuously encodes evolving real-time live content fused with behavior signal through residual quantization; (ii) A **Time-Aware Gated Attention** mechanism that explicitly models temporal dynamics for timely decision making; (iii) An efficient decoder-only generative architecture enhanced with **Sequential MTP** and **QK Norm** for stable training and accelerated inference; (iv) A **Unified Multi-Objective Alignment Framework** reinforces policy optimization for personalized preferences. Extensive offline experiments demonstrate the outperformance of our model. We have deployed OneLive in Kuaishou's live-streaming recommendation system, where online A/B tests show significant gains on multiple core business indicators, validating its effectiveness and practicality in real-world industrial scenarios.

1 Introduction

In recent years, industrial recommender systems [1, 17] have played a pivotal role across a wide range of online applications, including e-commerce [43], content distribution [1], advertising [38] and local services [33]. Their primary objective is to efficiently and accurately match users' preferences with relevant content under large-scale, high-concurrency real-world conditions, thereby enhancing user experience while maximizing key business metrics.

However, the relentless pursuit of higher accuracy has driven the evolution of recommendation models toward increasingly complex and sophisticated system architectures:

- **Cascade Architecture**: Recommender systems are required to perform personalized matching and ranking over massive candidate pools within an extremely short latency. To meet this demand, the industry has long adopted a cascade architecture as ***Retrieval - Pre-Ranking - Ranking***. However, this funnel-shaped framework imposes fundamentally misaligned objectives and constraints across stages. ***Retrieval*** aims to efficiently narrow down billions of items to a few thousand relevant candidates, prioritizing coverage and diversity under strict computational budgets. ***Ranking*** focuses on accurately estimating user preference for each candidate, where precision and multi-objective optimization are paramount. This misalignment often leads to locally optimal but globally suboptimal results, since the performance ceiling of downstream stages is inherently capped by the quality and diversity of candidates passed from upstream. Once superior items are prematurely filtered out in early stages, an irrecoverable information bottleneck is created, resulting in suboptimal recommendation and low efficiency in model iteration.
- **Fragmented Computational Graph**: Typical industrial recommender architecture comprise a heterogeneous mix of components, including sparse embedding lookups, sequential modeling and feature interaction modules, which results a highly irregular computational graph. This graph is dominated by memory accesses and data pipeline transfer, which fragment the execution flow. Consequently, the system becomes memory-bandwidth bound rather than compute bound, as the **Model FLOPS Utilization (MFU)** is often extremely low due to the ratio of effective floating-point operations in time consumption is lower than that of memory access and synchronization.

As the Transformer architecture has become the maintain standard in LLM [19, 35], which has demonstrated consistent gains with scaling, recommender systems are also showing a trend of converging towards th Transformer paradigm [28, 37, 39]. The core value of Transformers lies in scalability and computational efficiency. On the one hand, Transformer provides a unified modeling interface, which can organize heterogeneous signals from multiple sources like attribute features and behavior contexts into token sequences, enabling representation learning and interaction modeling within a single framework. On the other hand, Transformers are built around attention and FFN, whose computation is dominated by

† Equal Contribution.
‡ Jiangxia Cao is Corresponding Author.
∗ Ruochen Yang is an internship at Kuaishou. He is now at Institute of Information Engineering, Chinese Academy of Sciences.

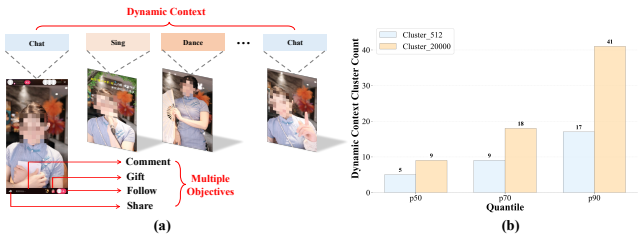


Figure 1: (a) Over a live-streaming lifecycle, the author exhibits diverse content types, accompanied by multi-objective user interactions. (b) Clustering 30-second segments across entire live-streamings and report the mean number of clusters to quantify continuous content dynamics.

large-scale matrix operations, thereby improving MFU through parallelization structure and obtaining benefits by scaling model capacity and training data.

Following this paradigm, OneRec [6] and a series of related **Generative Recommendation (GR)** methods [4, 10, 33, 42] have been proposed. These large recommendation models share a common theme that they advocate using a single and unified model architecture to accomplish end-to-end recommendation, together with preference alignmnet and engineering efforts for high throughput deployment. Concretely speaking, they tokenize business entities via residual quantization, perform autoregressive, session-wise generation with a unified Transformer backbone, and further align the model to fine-grained preference signals through reinforcement learning. By removing the systemic overhead of traditional multi-stage cascaded pipelines, this line of work achieves a more favorable cost structure and stronger objective consistency, and has consequently attracted substantial interest from industry [34, 38].

However, despite their general effectiveness, such methods cannot be directly applied to live-streaming recommendation without careful adaptation, owing to the inherent complexity of the live-streaming scenario:

- **Dynamic Content**: Unlike short videos or image-text contents that are fixed after users' upload, live-streaming content evolves continuously throughout the broadcast. Correspondingly, author activities, real-time user feedback and the overall interactive atmosphere all undergo substantial changes over time. For example, an author may frequently switch among chat, sing and dance like Figure 1(a). This dynamic nature challenges the standard tokenization paradigm in GR, where an item's static content can be encoded once into a stable representation for repeated use. In contrast, such a static tokenization paradigm fails to accommodate the dynamic nature of live-streaming, as the real-time content clustering statistics of the live-streaming shown in Figure 1(b). This necessitates dynamic modeling and tokenization with real-time content understanding and update to track live-streaming content drift.
- **Limited Lifecycle**: Live-streaming exhibit an inherent lifecycle constraint, where the full progression from initiation, growth, peak, decline to termination is typically confined within a short time window. This results in a strictly time-constrained candidate pool, where only ongoing live-streamings are eligible for recommendation, rather than the persistent content inventory in conventional platforms. A live-streaming must be promoted within its golden exposure window to realize its full value. Consequently, live-streaming GR must prioritize the validity of generated candidates throughout their entire lifecycle.
- **Real-Time Response Constraint**: Live-streaming recommendation operates in a high concurrent and critical latency online environment, where the candidate pool evolves rapidly and user queries arrive at extremely high frequency. This imposes stringent requirements on the live-streaming GR model, which must maintain competitive prediction accuracy while achieving efficient inference with low latency and sustained throughput.
- **Multiple Objectives**: Live-streaming recommendation is a multi-objective task with rich user feedback signals, including click, share, follow and gift in Figure 1(a). Users also exhibit distinct preferences across various forms of engagement and consumption behaviors, resulting in pronounced heterogeneity. Therefore, preference alignment over multi-objective signals is essential for live-streaming GR, requiring the integration of diverse supervision signals into a unified generative training pipeline.

To this end, we propose **OneLive**, a dynamically unified generative framework for live-streaming recommendation. Composed of a dynamic tokenizer, an efficient generative architecture, and a multi-objective alignment module, OneLive effectively addresses the unique challenges of live-streaming scenarios. To the best of our knowledge, OneLive is the first unified generative recommendation framework that has been successfully deployed on a large-scale, real-world live-streaming platform. We summarize the main contributions of this work as follows:

- We propose a novel **Dynamic Tokenizer** tailored to the inherently evolving nature of live-streaming content. By capturing real-time content information and fusing it with user behavioral signals, together with residual quantization, our method enables effective author representation learning and compression driven by online recommendation signals.
- We propose a **Time-Aware Gated Attention** mechanism to model the strict timing constraints imposed by the limited lifecycle of live-streaming. It explicitly incorporates the evolving patterns and dynamic user feedback into interaction modeling and decision making, thereby meeting the stringent timeliness requirements of live-streaming distribution.
- We propose a **Sequential Multi-Token Prediction (MTP)** mechanism to accelerate inference in the decoder-only GR architecture, and further introduce **QK Normalization** as a key component to ensure stable training and reliable online serving.
- We propose a **Unified Multi-Objective Alignment Framework** powered by reinforcement learning with an ensemble ranking reward model. This framework explicitly fuses personalized multi-objective and aligns heterogeneous user preferences to accommodate diverse behaviors in live-streaming scenarios.
- We conduct extensive offline experiments to validate our framework and individual modules. We have deployed OneLive on Kuaishou's live-streaming recommendation system, and large-scale online A/B tests show significant and consistent improvements on core business metrics, verifying its practical value in real industrial scenarios.

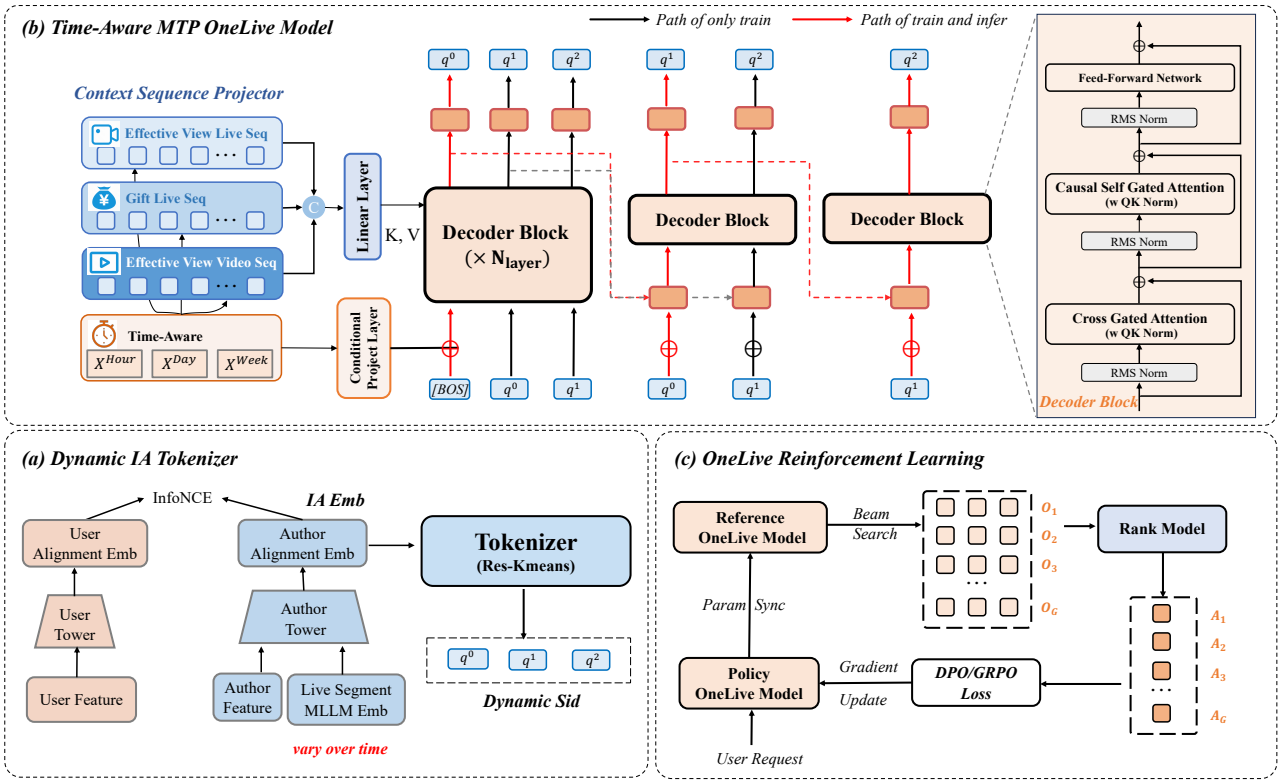


Figure 2: Overall framework of OneLive.

2 Methodology

In this section, we propose OneLive, an end-to-end generative framework for live-streaming recommendation. The overall architecture of our framework is illustrated in Figure 2.

2.1 Dynamic Tokenizer

As commonly practiced in existing GR methods [6, 28], the generation of item embeddings followed by semantic quantization constitutes a core operation in tokenization pipelines. This two-step process balances the commonalities and distinct characteristics among candidate items, and effectively preserves essential information while controlling vocabulary size. Conventionally, this pipeline focuses on items with fixed content after upload, where it first captures the comprehensive content of such items to generate item embeddings, and subsequently quantizes these item embeddings into a set of multi-level codes, referred to as Semantic IDs (SIDs), via methods such as RQ-VAE [16] or Res-Kmeans [23]. Some alternative approaches [31, 34] further incorporate behavioral information as collaborative signals into content understanding, thereby enhancing the quality of item embedding generation.

However, the dual dynamics of live-streaming scenarios pose non-trivial challenges to the direct application of these conventional pipelines: (i) ***Dynamic streaming content***: Authors engage in diverse activities (*e.g.*, chatting, singing and dancing) over time, causing real-time changes in live-streaming content. (ii) ***Dynamic user behavior***: Temporal changes in live-streaming content directly affect users' preferences commenting, gifting and other interactions. Therefore, it is crucial for a tokenizer to address the dual dynamics issue. Consequently, we propose a unified dynamic tokenizer in OneLive, which jointly models real-time live-streaming content and users' immediate behavioral feedback during the item embedding generation stage.

2.1.1 ***Semantic and Collaborative Dynamic Alignment.*** Live-streaming real-time content is multimodal, encompassing visual, auditory, and textual information. However, content-only embeddings lack collaborative signals and suffer from low discriminability, such that semantically similar clips may yield nearly identical representations and thus fail to distinguish different streamers. Naive embedding fusion with author side information fails to capture evolving popularity or cohort-specific preferences [34]. Alternatively, directly injecting collaborative signals into MLLM pretraining [31, 32] is ill-suited for live streaming. These approaches rely on static, pre-collected behavior data, while user-author collaborative patterns evolve rapidly and unpredictably in real-time, which cannot be captured by offline pretraining.

To address this dilemma, we introduce a two-stage paradigm consisting of *dynamic content understanding* and subsequent *real-time collaborative post-alignment*, which jointly generates the final item embedding for real-time live-streaming segments.

Dynamic Content Understanding. To model cross-modal live-streaming content, we employ a MLLM to generate representation embeddings. Given the trade-off between contextual integrity and

Table 1: The Code Utilization Rate (UR) and Collision Rate (CR) under diverse codebook settings. The embeddings are collected from about 4 million authors. The results of best combination are boldfaced. Metrics definitions are provided in Appendix C.1.

Type	Method	Size	UR ↑				CR ↓	
			L_0	L_1	L_2	$L_0 \times L_1$	SID	Author
MLLM	Res-Kmeans	(512, 512, 512)	100.00%	99.61%	97.66%	53.83%	28.10%	63.83%
		(8192, 8192, 8192)	88.99%	63.93%	53.10%	2.34%	3.56%	9.47%
IA	RQ-VAE	(512, 512, 512)	100.00%	100.00%	100.00%	74.41%	15.76%	39.76%
		(8192, 8192, 8192)	100.00%	100.00%	100.00%	1.16%	8.54%	22.78%
	Res-Kmeans	(512, 512, 512)	100.00%	100.00%	100.00%	93.98%	9.72%	23.03%
		(8192, 8192, 8192)	**100.00%**	**100.00%**	**100.00%**	**4.50%**	**0.66%**	**1.76%**

online latency constraints, we balance these demands by adopting a 30-second sliding window to dynamically update live-streaming content embeddings [1] in practice. We leverage a large LLM to curate a dataset of content clips, and further fine-tune a lightweight MLLM through knowledge distillation [21], which continuously ingests real-time segmented content to generate live-streaming dynamic embeddings. *Real-time Collaborative Post-alignment.* Building upon the dynamic content embeddings obtained above, we further conduct real-time post-alignment with collaborative signals using a dual-tower architecture. This stage explicitly integrates instant user interaction feedback (e.g., commenting, gifting, clicking) to complement semantics-only representations, thereby alleviating discriminability issues and adapting to the highly dynamic user-author collaboration patterns.

Specifically, in the author tower, we fuse the author's static attributes $x^{AId} \in \mathbb{R}^d$ with dynamic content features. The latter includes real-time content embedding $x^{MLLM}_{30s} \in \mathbb{R}^d$ derived from the most recent 30-second window to display instant status, and long-term average pooling embedding $x^{MLLM}_{pooling} \in \mathbb{R}^d$ which reflects author's thematic commonalities. We employ a gating mechanism for representation fusion on the author side:

$$x^{MLLM} = \text{MLP}(x^{MLLM}_{30s} \oplus x^{MLLM}_{pooling}), \tag{1}$$

$$x^{Author} = \lambda x^{AId} + (1-\lambda) x^{MLLM}, \tag{2}$$

where λ is a trainable weight to balance heterogeneous inputs.

The user's attributes $x^{UId} \in \mathbb{R}^d$ are processed through the user tower, and then we peform user-author alignment based on the interaction samples in the data streaming:

$$x^{User} = \text{MLP}(x^{UId}), \tag{3}$$

$$\mathcal{L}_{Alignment} := \text{In-Batch-Softmax}(x^{User}, x^{Author}). \tag{4}$$

The output embedding of the author tower x^{Author} preserves author attributes and real-time content generalization via gated fusion, and incorporate collaborative signals through dual-tower interaction supervision. We obtain the final semantically and collaboratively aligned item embedding for live-streaming GR, termed IA Embedding $x^{IA} := x^{Author}$, for downstream quantification tasks.

2.1.2 ***Residual Quantization.*** To pursue better scalability and more explicit hierarchical semantics, we opt for Res-Kmeans over RQ-VAE for residual quantization. Specifically, the IA Embedding is quantized in a coarse-to-fine manner, where K-means is applied iteratively to the multi-level residuals $\mathcal{R}^l$ to heuristically construct the codebook at each layer:

$$q^l = \arg\min_k \|c^l_k - x^l\|, \tag{5}$$

where $c^l_k \in \mathcal{C}^l = \text{Kmeans}(x^l, N_l)$ is the codebook derived from N_l centroids obtained through clustering, and $x^{l+1} = x^l - c^l_{q^l}$ is the residual of l layer as the input for the $l+1$ layer. By iterating to obtain the quantization code of each layer q^l for T times, we can get the SID corresponding to the author in real time $\{q^0, q^1, \ldots, q^T\}$.

We compare the quantization quality under different settings, with results reported in Table 1. IA Embeddings achieve higher codebook utilization and lower code collision rates than MLLM Embeddings, owing to refinement via collaborative post-alignment and supervision from user-item interaction signals. Similarly, Res-Kmeans quantization outperforms RQ-VAE with improved codebook utilization and fewer collisions under the same settings, and increasing the codebook size effectively alleviates code collisions. We thus employ Res-Kmeans with $T = 3$ layers and codebook size $N_l = 8192$ for semantic quantization.

2.2 Generative Recommender

OneLive adopts a lightweight and time-aware architecture. Built upon a standard decoder-only backbone, we model temporal dynamics via a gated attention mechanism, improve inference efficiency through sequential multi-token prediction, and enhance training stability with QK Norm.

2.2.1 ***Basic Decoder-Only Architecture.*** In industrial live-streaming recommendation, modeling user preferences over authors relies on integrating diverse behavior sequences (e.g., live-streaming effective viewing S^{Eff}, gifting S^{Gift}, short-video viewing S^{Video}) to capture fine-grained interests. However, encoding each sequence respectively with Encoder-Decoder architectures like T5 [27], TIGER [28] incurs substantial overhead. The core issues is that the sequences are numerous and lengthy, with total length far exceeding the target streamer SID (*e.g.*, $3{\times}500 \gg 3$, where $n = 3, l = 500$ denote the number and per-sequence length). As a result, vast majority of computation goes to the encoder stage, wasted on redundant or noisy interactions in long sequences rather than target SID generation.

Inspired by [42], we adopt a lazy decoder-only architecture that avoids explicit encoding of lengthy user sequences, while maintaining model capacity comparable to conventional encoder-decoder structures for complex user-author interactions.

Specifically, we project user's multiple historical sequences into an embedding space, which serves as key-value contexts for generation:

$$S = \{S^{Eff}, S^{Gift}, S^{Video}, \dots\}_n, \tag{6}$$

$$x_i = \text{Concat}(x_i^{\text{AId}}, x_i^{\text{IA}}, \dots) = \text{Emb}\left(\mathcal{S}[i]\right) \in \mathbb{R}^d, \tag{7}$$

$$E = \text{MLP}(\{x_0, x_1, \dots, x_{|S|}\}), \tag{8}$$

where $E \in \mathbb{R}^{|S| \times d}$ is the encoding of user behavioral signals, and $|S| = n \times l$ is the length of concatenated sequence.

The Decoder-Only architecture is composed of L stacked transformer blocks, with three main components: cross-attention, self-attention and feed-forward network (FFN). Formally, the computation of the l-th layer is:

$$h^l = h^{l-1} + \text{CrossAttn}(q = \text{RMSNorm}(h^{l-1}), k = E, v = E), \tag{9}$$

$$h^l = h^l + \text{SelfAttn}(\text{RMSNorm}(h^l)), \tag{10}$$

$$h^l = h^l + \text{FFN}(\text{RMSNorm}(h^l)), \tag{11}$$

where h^l denotes the hidden state of each layer. For the first layer, $h^0 = \text{Emb}(\{[\text{BOS}], q^0, q^1\}) = \{x_{[\text{BOS}]}, x_{q^0}, x_{q^1}\} \in \mathbb{R}^{3 \times d}$ under the teacher-forcing training paradigm.

2.2.2 ***Temporal Dynamics Conditioning.*** Live-streaming recommendation is a typical time-constrained distribution scenario that aligns with the inherent limited lifecycle of live-streamings, where only ongoing live broadcasts from online authors are eligible for recommendation and system distribution. To address this strict temporal constraint, we model temporal dynamics from three perspectives: *historical sequence temporal perception*, *generation anchor temporal perception*, and *attention-gated temporal perception*.

Historical Sequence Temporal Perception. We inject multi-granular time biases (e.g., hour, day, week) associated with each historical item x_i (Eq. 7) as temporal signals to construct time-aware sequential representations:

$$x_{i\_ta} = x_i + \text{MLP}\left(\text{Concat}(x_i^{\text{Hour}}, x_i^{\text{Day}}, x_i^{\text{Week}})\right), \tag{12}$$

which are further mapped into the time-aware context embedding E_{ta} following Eq. 8.

Generation Anchor Temporal Perception. As the [BOS] token is the autoregressive generation anchor in our lazy decoder-only framework, we augment [BOS] with multi-granular temporal features that mark the instantaneous query time of the user, enabling time-aware conditional generation:

$$x_{[\text{BOS}]\_ta} = x_{[\text{BOS}]} + \text{MLP}\left(\text{Concat}(x_q^{\text{Hour}}, x_q^{\text{Day}}, x_q^{\text{Week}})\right), \tag{13}$$

where x_q^* denotes the embedding of temporal features of the query moment.

Attention-Gated Temporal Perception. We equip each SID generation step with adaptive context importance selection by introducing gated attention [25] into the decoder. This mechanism dynamically modulates attention weights over both user historical encoding and the preceding decoded sequence via learnable gating scores. It is applied to both cross-attention (Eq. 9) and self-attention (Eq. 10):

$$\text{Score}(X) = \sigma(XW_\theta), \tag{14}$$

$$O = \left(\text{MultiHeadAttn}(XW_Q, X'W_K, X'W_V) \odot \text{Score}(X)\right) W_O, \tag{15}$$

where $X' = E$ for cross-attention and $X' = X$ for self-attention, $W_\theta \in \mathbb{R}^{d \times d}$ denotes the gating projection parameter, and $\sigma(\cdot)$ is the Sigmoid activation. The gating scores are applied via element-wise multiplication after multi-head attention concatenation, allowing the model to adaptively emphasize temporally relevant context.

2.2.3 ***Sequential Multi-Token Prediction.*** The standard autoregressive decoder is trained via the next-token prediction (NTP) mechanism, where cross-entropy loss is adopted with teacher-forced ground-truth SID codes $\{q^0, q^1, q^2\}$. The corresponding loss function is formally defined as follows:

$$\mathcal{L}_{NTP} := -\sum_{i=0}^{2} \log p(q^i | [\text{BOS}], q^{<i}, E), \tag{16}$$

$$p(q^i | [\text{BOS}], q^{<i}, E) = \text{Softmax}(\text{MLP}(h_i^L)). \tag{17}$$

During the inference phase, beam search is utilized to perform greedy autoregressive generation. The predicted next author's SID $p(Q \mid E)$ is formally formulated as

$$p(Q|E) = p(q^0|[\text{BOS}], E) \cdot p(q^1|[\text{BOS}], q^0, E) \cdot p(q^2|[\text{BOS}], q^{0:1}, E). \tag{18}$$

A critical limitation of the aforementioned standard inference approach is inefficiency: in practice, upon receiving a user request, a set of q^0 are generated using the given [BOS] token, and so forth—subsequently feeding the inferred set of each preceding SID token into the decoder to generate the next, i.e., q^1 from q^0 and then q^2 from q^1. As a result, the inference batch size for q^0 is relatively small, while the inference batch sizes for q^1 and q^2 would be much larger—this computational overhead expands rapidly when a large beam size is adopted. Meanwhile, we observe that SID1 and SID2 are residual components of SID0, and their prediction tasks rely on the conditional probability of the already inferred preceding SIDs, making these two subtasks relatively straightforward.

To address the aforementioned computational bottleneck while leveraging this key observation, we propose a **Sequential Multi-Token Prediction (Sequential MTP)** mechanism inspired by [19]. As shown in Figure 2(b), our model have three sequentially connected decoders with a shared SID embedding layer. The first MTP module, referred to as the main-decoder, retains the full L layers transformer blocks similar to the standard setting. Subsequent MTP modules adopt a lightweight single-layer decoder block, which takes as input the concatenation of the previous MTP module's hidden states and the embedding of the immediately preceding SID:

$$h_i^k = \text{MLP}(\text{Concat}(\{h_i'^{k-1}, \text{Emb}(q^i)\})), \tag{19}$$

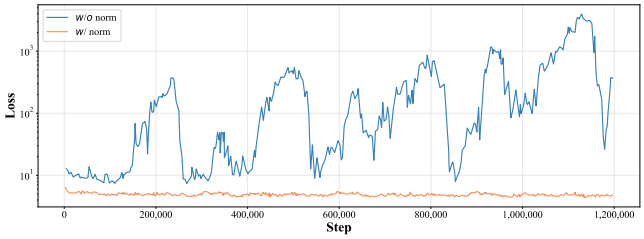
$$h_{i:2}'^k = \text{MTP}_k(h_{i:2}^k). \tag{20}$$

To reuse the KV cache generated during q^0 inference, the decoder-lite module shares the parameters of the main-decoder's first layer (except for the FFN component in Eq.11). The final training task is the weighted fusion of losses from multiple MTP modules:

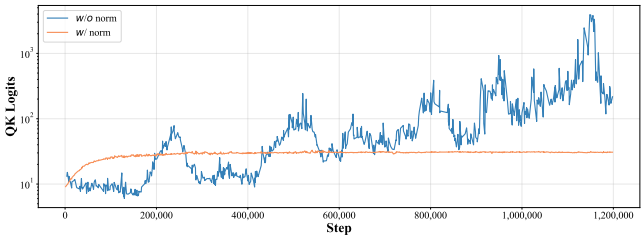
$$\mathcal{L}_{MTP} := w_0 \mathcal{L}_{\text{MTP}_{main}} + w_1 \mathcal{L}_{\text{MTP}_1} + w_2 \mathcal{L}_{\text{MTP}_2}, \tag{21}$$

$$\mathcal{L}_{\text{MTP}_k} := -\log p(q^{k:2} | [\text{BOS}], q^{<2}, E), \tag{22}$$

where w_k is smaller for subsequent MTP modules than for the main-decoder.



(a) Trend comparison of loss.



(b) Trend comparison of max QK logits.

Figure 3: The influence of QK Norm on training stability, which prevents the explosion of loss and max QK logits.

During inference, we use the main decoder to infer q^0, followed by lite decoder for q^1 and q^2. Owing to parameter sharing between modules allows the fully utilization of KV cache for autoregressive generation, our Sequential MTP mechanism is expected to achieve higher inference efficiency than the standard decoder-only.

2.2.4 ***QK Norm Optimization.*** Nevertheless, when further increasing model depth, we observe training instability, as the loss fluctuation shown Figure 3(a). After prolonged training, the loss occasionally spikes and may fail to recover. Diagnostics indicate that the QK logits QK^T in deep attention layers grows substantially over training, exhibiting logit explosion. Since we implement bf16 mixed-precision training, the effective numerical precision and dynamic range of the QK logits are limited, with the resulting numerical errors and scale drift can be amplified through deep stacking, progressively expanding the logits [24]. Once fed into the Softmax, the attention distribution becomes approaching a one-hot vector, which markedly attenuates gradients and can ultimately collapse training.

To improve training stability, we introduce QK Norm [12]. Specifically, before computing the attention QK logits, we apply RMSNorm to the Query and Key vectors to suppress scale drift across depth and throughout training, making the resulting logits closer to a scale-insensitive similarity measure:

$$\text{Attn}(Q,K,V) = \text{Softmax}(\frac{\text{RMSNorm}(Q)\text{RMSNorm}(K)^T}{\sqrt{d}})V. \quad (23)$$

This normalization effectively constrains the magnitude of the logits and reduces the risk of softmax saturation caused by excessively large values, thereby improving training stability. As shown in Figure, with QK Norm enabled, the occasional loss spikes observed during prolonged training disappear.

2.3 Reinforcement Learning

Training OneLive solely via next author prediction on offline logs amounts to behavior cloning, which optimizes the model to mimic historical decision patterns of the online serving system. Although such a paradigm maintains consistency with past policies, it inherently restricts the model from breaking through the performance ceiling of the deployed system [6]. To overcome this limitation, we introduce an explicit reward model and conduct on-policy reinforcement learning, which directly optimizes the generation policy in the generation space using reward signals that reflect users' potential preferences. Crucially, the RL objective drives the model to concentrate its generative probability on high-reward candidates, so that the top item set generated by beam search becomes consistent with users' potentially preferred rankings, directly improving the quality of the head generation results at inference time.

2.3.1 ***Reward Model.*** The reward for each generated candidate must reflect holistic alignment with users' multi-objective preferences in live-streaming scenarios, including diverse feedback signals such as click, long-view and gift. As emphasized in our motivation, live-streaming recommendation exhibits strong user heterogeneity in engagement behaviors, demanding flexible and user-adaptive integration of multiple objectives rather than rigid heuristics. A naive baseline is to fuse multiple task-specific scores (e.g., CTR, LVTR, GTR) predicted by a ranking model using fixed handcrafted weights to construct the reward [4, 10]. Such heuristic fusion has two critical drawbacks, in that fixed weights cannot adapt to user-specific preference heterogeneity and the overall scheme lacks flexibility for online tuning.

To address these issues and enable multi-objective alignment for generative training, we employ the unified multi-objective ensemble ranking model Pantheon [2] as our reward model. This model is optimized via a standard additive weighting mechanism to produce a single unified ranking score that inherently balances diverse objectives while adapting to user specific preference heterogeneity:

$$Score = \text{Pantheon}(\text{User}, \text{Author}), \quad (24)$$

$$\mathcal{L}^{XTR}_{Pantheon} := y^{XTR}\log(Score) + (1 - y^{XTR})\log(1 - Score), \quad (25)$$

$$\mathcal{L}_{Pantheon} := \sum_{XTR}^{CTR,LVTR,GTR,\ldots} w^{XTR}\mathcal{L}^{XTR}_{Pantheon}. \quad (26)$$

We directly use the multi-objective ensemble score $r := Score$ as the reward of each candidate author for corresponding user.

2.3.2 ***Optimize Strategy.*** We explore different preference optimization strategies, including Direct Preference Optimization (DPO) [26] and Group Relative Policy Optimization (GRPO) [9]. In both cases, a set of response $\{o_1, o_2, \ldots, o_G\}$ for each query q is generated through a reference model, which periodically synchronizes parameters from the offline streaming training policy model. The candidate authors in response are then scored by the reward model to obtain reward values $\{r_1, r_2, \ldots, r_G\}$, and then the advantage scores are obtained via normalization $A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$. Subsequently, we perform policy optimization:

- DPO: We choose the authors with highest advantage as the positive sample o_{pos}, and the lowest one as the negative sample o_{neg}.

Table 2: The overall performance comparison of different models on live-streaming offline dataset. The best results are boldfaced and the second-best results are underlined, with the relative improvements denoted as *Imprv.*↑.

Models		LongView				Click			
		HR@64	MRR@64	HR@128	MRR@128	HR@64	MRR@64	HR@128	MRR@128
Traditional	SASRec	0.2525	0.0665	0.3047	0.0671	0.2394	0.0623	0.2924	0.0629
	KuaiFormer	0.3181	0.0907	0.3762	0.0913	0.3145	0.0890	0.3750	0.0897
	GNN	0.4720	0.1548	0.5465	0.1556	0.4565	0.1469	0.5319	0.1478
Generative	TIGER	0.2847	0.0795	0.3395	0.0801	0.2877	0.0785	0.3442	0.0791
	OneRec	0.5967	0.2836	0.6347	0.2840	0.5802	0.2667	0.6209	0.2671
Ours	**OneLive**	**0.6887**	**0.3213**	**0.7388**	**0.3218**	**0.6720**	**0.3046**	**0.7246**	**0.3052**
	Imprv. ↑	**15.42%**	**13.29%**	**16.40%**	**13.31%**	**15.82%**	**14.21%**	**16.70%**	**14.26%**

Based on these samples, the policy model is optimized to align user preferences:

$$\mathcal{L}_{\text{DPO}} := -\sigma(\lambda \log \frac{\pi_\theta(o_{pos}|q)}{\pi_{\theta ref}(o_{pos}|q)} - \lambda \log \frac{\pi_\theta(o_{neg}|q)}{\pi_{\theta ref}(o_{neg}|q)}). \quad (27)$$

- GRPO: We use the relative advantages of candidate authors to optimize the policy model, enabling it to learn better responses:

$$\mathcal{L}_{\text{GRPO}} := \frac{1}{G}\sum_{i=1}^{G} \min(\frac{\pi_\theta(o_i|q)}{\pi_{\theta ref}(o_i|q)}A_i, \text{clip}(\frac{\pi_\theta(o_i|q)}{\pi_{\theta ref}(o_i|q)}, 1-\epsilon, 1+\epsilon)A_i). \quad (28)$$

We incorporate RL into our offline streaming training. To ensure the stability and sonsistency of the training process, we only sample 1% query per pass to run the RL policy. The final loss will be a weighted fusion of the MTP and RL loss:

$$\mathcal{L}_{OneLive} := \mathcal{L}_{MTP} + w\mathcal{L}_{RL}. \quad (29)$$

3 Experiments

In this section, we present extensive offline experiments to demonstrate the performance improvement of OneLive and its related components compared to baselines in live-streaming scenarios. We also conduct rigorous A/B tests to verify the effectiveness of OneLive in real-world online deployment. In addition, we provide case studies to offer further analysis of the gain sources of OneLive. Details of experiment settings refer to Appendix B.1.

3.1 Offline Experiment

3.1.1 ***Overall Performance.*** As results shown in Table 2, our proposed model OneLive consistently outperforms all existing baselines, with the maximum scale improvements on click records reaching up to 16.70% in HR@128 and 14.26% in MRR@128. It also surpasses the best baseline by at least 13.29% across other indicators. These results strongly validate the effectiveness of our generative recommendation framework with dynamic awareness in live-streaming recommendation, rather than simply applying the existing generative methods (*e.g.* OneRec) to switch scenarios. OneLive not only rivals but even exceeds carefully engineered industrial architectures based on ANN retrieval (*e.g.* KuaiFormer and GNN), thereby offering superior support for real-time content understanding and interaction prediction. The detailed analysis of the overall experiments is presented in Appendix B.2.

Table 3: The ablation experiments conducted in component stacking under click records. The best performances of each component are boldfaced.

Variants	ACC@all	Imprv. ↑	HR@128	Imprv. ↑
OneRec				
+ 3 × 512 MLLM	0.613	-	0.621	-
Tokenizer				
+ 3 × 8192 MLLM	0.592	-3.42%	0.643	+3.54%
+ 3 × 512 IA	0.616	+0.49%	0.640	+3.06%
+ 3 × 8192 IA	0.608	-0.82%	0.684	+10.14%
Architecture				
+ Standard Dec-Only	0.618	+0.82%	0.695	+11.92%
+ Lazy Dec-Only	0.621	+1.31%	0.710	+14.33%
+ MTP	0.619	+0.98%	0.708	+14.01%
Temporal				
+ Gated-Attn	0.619	+0.98%	0.710	+14.33%
+ TA Gated-Attn	0.630	+2.77%	0.717	+15.46%
OneLive				
+ Full	0.641	+4.57%	0.725	+16.75%

3.1.2 ***Component Ablation.*** We conduct component ablation and replacement experiments on three core modules of OneLive. We regard OneRec as our baseline, which adopts an Encoder-Decoder architecture to predict 3 × 512 MLLM code to adapt live-streaming scenario. We incrementally stack each component, and report the metrics ACC@all of training and HR@128 of inference, with results shown in Table 3. We summarize key observations as follows:

- **Tokenizer**: Results on inference of multiple tokenizer demonstrate two conclusions: on the one hand, IA Code outperforms MLLM Code by incorporating behavior signals to obtain more discriminative SIDs; on the other sid, increasing the codebook size also yields performance gains. Both changes can reduce collision ratio thereby enhancing the individual expression of SIDs. Besides, we observe the misalignment between training and inference metrics, which might due to coupled data simplifies the model's learning but results in degraded accuracy and generalization during inference.
- **Architecture**: Results of architecture show that, under similar parameter quantity, Decoder-Only allocates more computation

Table 4: The comparison of QPS improvement and computational graph execution latency.

Number of Layers	Variants	QPS Imprv. ↑	Latency (ms, Avg)	Latency (ms, P99)
6	Lazy Dec-Only	-	6.82	13.34
	+ MTP	+62.0%	2.97	5.04
3	Lazy Dec-Only	-	5.05	8.48
	+MTP	+31.8%	2.25	3.62

Table 5: The ablation experiment of reinforcement learning.

Beam Top	Model	Reward			
		LVTR	CTR	WTR	GTR
Top64	OneLive	0.03808	0.04883	0.00536	0.00127
	+ DPO	0.03680	0.04723	0.00513	0.00123
	+ GRPO	0.04070	0.05042	0.00549	0.00136
Top256	OneLive	0.03066	0.04092	0.00473	0.00113
	+ DPO	0.03131	0.04142	0.00468	0.00114
	+ GRPO	0.03405	0.04346	0.00494	0.00118

to the decoding process compared to Encoder-Decoder architecture, thereby enhancing the model's generation capability. Furthermore, the similar implementation of Lazy Decoder-Only improves upon Naive Decoder-Only by introducing explicit sequence crossover, enabling adaptive context aggregation during generation. The introduction of the Sequential MTP mechanism maintains comparable generation performance, while achieving substantial inference acceleration, characterized by a **31.8%** increase in single-machine QPS and a **55.45%** reduction in computational graph execution latency, as the results shown in Table 4. Our hardware utilization is also substantially improved, achieving a MFU of **22.78%** on the L20 GPU (compared to only **2.56%** for the online ranking model).

- **Temporal**: Results of temporal modeling show that simple gated attention does not bring obvious benefits. However, after incorporating temporal side information into sequence modeling and prompt design, the model achieves adaptive attention gating based on temporal awareness. This leads to reasonable distribution of living authors, as the improvement of the validity rate by **14.29%** during inference.

3.1.3 ***RL Ablation.*** We present the ablation experiments of reinforcement learning based on OneLive in Table 5. The results show that compared with the baseline, GRPO achieves consistent performance and system recognition improvements across different beam size settings, while DPO only filters better candidates that align with user preferences when the beam size is large. This might because that GRPO makes use of all the responses within the group for advantages calculation and gradient optimization, which provides richer and more robust feedback signals. However, DPO optimizes directly based on limited feedback from positive and negative sample pairs, which increases the risk of being affected by noise in user preferences under small baem size.

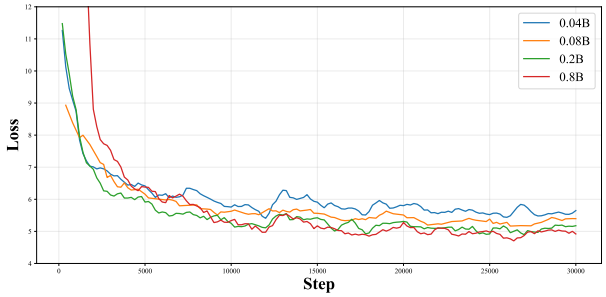


Figure 4: Loss curves under parameter scaling.

Table 6: Online A/B test performance in live-streaming services of Kuaishou and Kuaishou Lite.

Application	Exposure	CTR	Click Count	Watch Time	Follow
Kuaishou	+1.32%	+0.41%	+1.73%	+0.58%	+1.36%
Kuaishou Lite	+1.96%	+0.72%	+2.70%	+0.41%	+2.07%

3.1.4 ***Parameter Scaling.*** To investigate the suitability of scaling law in live-streaming generative recommendation model, we conduct a model scaling experiment. We train model across different parameter scales and visualize the loss curves in Figure 4. The results exhibit that, as model size increases, the performance shows a stable upward trend, with loss consistently decreases. However, the gains exhibit a diminishing marginal returns effect, as the loss decrease brought by further scaling based on larger parameter scale gradually slows down. Considering the online deployment constraints, we finally deploy 0.08B version for online service.

3.2 Online A/B Test

We deploy OneLive on Kuaishou and Kuaishou Lite, providing servies to hundreds of millions of users and millions of authors on two major live-streaming platforms, to observe the real effectiveness of our model's performance for online business. We launch our optimal combo model version to the production environment, and conduct continuous online observation for nearly one month. The results are summarized in Table 6.

OneLive achieves significant gains in user penetration within the live-streaming recommendation scenario. Across two real world applications, both exposure and CTR show substantial improvements at the page level, which validates the significant advantage of our unified generative architecture over the online recommendation pipeline. Besides, multiple behavior indicators such as click, watch and follow, also increase noticeably. These results demonstrate that our model accurately captures user preferences and enables efficient content distribution.

4 Conclusion

In this paper, we propose dynamically unified generative framework OneLive tailored for live-streaming recommendation. We capture and quantify live-streaming real-time content using a dynamic tokenizer, perform temporal modeling with time-aware gated attention, achieve stable training and optimized inference through Sequential MTP and QK normalization, and align personalized preferences via multi-objective reinforcement learning. Extensive offline and online

experiments demonstrate the effectiveness of OneLive. Now OneLive has been deployed on Kuaishou App and serving 400 million users daily in live-streaming scenario, bringing significant business benefits. In the future, we will continue to explore the more effective and efficient live-streaming generative recommendation.

References

[1] Jiangxia Cao, Shen Wang, Yue Li, Shenghui Wang, Jian Tang, Shiyao Wang, Shuang Yang, Zhaojie Liu, and Guorui Zhou. 2024. Moment&Cross: Next-Generation Real-Time Cross-Domain CTR Prediction for Live-Streaming Recommendation at Kuaishou. *arXiv preprint arXiv:2408.05709* (2024).

[2] Jiangxia Cao, Pengbo Xu, Yin Cheng, Kaiwei Guo, Jian Tang, Shijun Wang, Dewei Leng, Shuang Yang, Zhaojie Liu, Yanan Niu, et al. 2025. Pantheon: Personalized multi-objective ensemble sort via iterative pareto policy optimization. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 5575–5582.

[3] Jiangxia Cao, Ruochen Yang, Xiang Chen, Changxin Lao, Yueyang Liu, Yusheng Huang, Yuanhao Tian, Xiangyu Wu, Shuang Yang, Zhaojie Liu, et al. 2025. Foresight Prediction Enhanced Live-Streaming Recommendation. *arXiv preprint arXiv:2512.06700* (2025).

[4] Ben Chen, Xian Guo, Siyuan Wang, Zihan Liang, Yue Lv, Yufei Ma, Xinlong Xiao, Bowen Xue, Xuxin Zhang, Ying Yang, et al. 2025. Onesearch: A preliminary exploration of the unified end-to-end generative framework for e-commerce search. *arXiv preprint arXiv:2509.03236* (2025).

[5] Jiaxin Deng, Dong Shen, Shiyao Wang, Xiangyu Wu, Fan Yang, Guorui Zhou, and Gaofeng Meng. 2023. ContentCTR: Frame-level live streaming click-through rate prediction with multimodal transformer. *arXiv preprint arXiv:2306.14392* (2023).

[6] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. Onerec: Unifying retrieve and rank with generative recommender and iterative preference alignment. *arXiv preprint arXiv:2502.18965* (2025).

[7] Jiaxin Deng, Shiyao Wang, Yuchen Wang, Jiansong Qi, Liqin Zhao, Guorui Zhou, and Gaofeng Meng. 2024. MMBee: Live Streaming Gift-Sending Recommendations via Multi-Modal Fusion and Behaviour Expansion. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4896–4905.

[8] Yi Fang, Wenjie Wang, Yang Zhang, Fengbin Zhu, Qifan Wang, Fuli Feng, and Xiangnan He. 2025. Reason4Rec: Large Language Models for Recommendation with Deliberative User Preference Alignment. *arXiv preprint arXiv:2502.02061* (2025).

[9] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. 2025. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature* 645, 8081 (2025), 633–638.

[10] Xian Guo, Ben Chen, Siyuan Wang, Ying Yang, Chenyi Lei, Yuqing Ding, and Han Li. 2025. OneSug: The Unified End-to-End Generative Framework for E-commerce Query Suggestion. *arXiv preprint arXiv:2506.06913* (2025).

[11] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.

[12] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. 2020. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 4246–4253.

[13] Minjie Hong, Yan Xia, Zehan Wang, Jieming Zhu, Ye Wang, Sihang Cai, Xiaoda Yang, Quanyu Dai, Zhenhua Dong, Zhimeng Zhang, et al. 2025. EAGER-LLM: Enhancing Large Language Models as Recommenders through Exogenous Behavior-Semantic Integration. In *Proceedings of the ACM on Web Conference 2025*. 2754–2762.

[14] Min Hou, Le Wu, Yuxin Liao, Yonghui Yang, Zhen Zhang, Changlong Zheng, Han Wu, and Richang Hong. 2025. A Survey on Generative Recommendation: Data, Model, and Tasks. arXiv:2510.27157 [cs.IR] https://arxiv.org/abs/2510.27157

[15] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.

[16] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 11523–11532.

[17] Rui Li, Pengyuan Gao, Haihan Li, Ling Chai, Shaohao Huang, and Ting Xie. 2025. A Bilateral Perspective for Modeling Real-Time Traffic Trends in Live-Streaming Recommendation. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 4142–4155.

[18] Xiaodong Li, Ruochen Yang, Shuang Wen, Shen Wang, Yueyang Liu, Guoquan Wang, Weisong Hu, Qiang Luo, Jiawei Sheng, Tingwen Liu, et al. 2025. FARM: Frequency-Aware Model for Cross-Domain Live-Streaming Recommendation. *arXiv preprint arXiv:2502.09375* (2025).

[19] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[20] Chi Liu, Jiangxia Cao, Rui Huang, Kai Zheng, Qiang Luo, Kun Gai, and Guorui Zhou. 2024. KuaiFormer: Transformer-Based Retrieval at Kuaishou. *arXiv preprint arXiv:2411.10057* (2024).

[21] Yueyang Liu, Jiangxia Cao, Shen Wang, Shuang Wen, Xiang Chen, Xiangyu Wu, Shuang Yang, Zhaojie Liu, Kun Gai, and Guorui Zhou. 2025. LLM-Alignment Live-Streaming Recommendation. *arXiv preprint arXiv:2504.05217* (2025).

[22] Yucheng Lu, Jiangxia Cao, Xu Kuan, Wei Cheng, Wei Jiang, Jiaming Zhang, Yang Shuang, Liu Zhaojie, and Liyin Hong. 2025. LiveForesighter: Generating Future Information for Live-Streaming Recommendations at Kuaishou. *arXiv preprint arXiv:2502.06557* (2025).

[23] Xinchen Luo, Jiangxia Cao, Tianyu Sun, Jinkai Yu, Rui Huang, Wei Yuan, Hezheng Lin, Yichen Zheng, Shiyao Wang, Qigen Hu, et al. 2025. Qarm: Quantitative alignment multi-modal recommendation at kuaishou. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 5915–5922.

[24] Haiquan Qiu and Quanming Yao. 2025. Why Low-Precision Transformer Training Fails: An Analysis on Flash Attention. *arXiv preprint arXiv:2510.04212* (2025).

[25] Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, et al. 2025. Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free. *arXiv preprint arXiv:2505.06708* (2025).

[26] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems* 36 (2023), 53728–53741.

[27] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.

[28] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.

[29] Jérémie Rappaz, Julian McAuley, and Karl Aberer. 2021. Recommendation on live-streaming platforms: Dynamic availability and repeat consumption. In *Proceedings of the 15th ACM Conference on Recommender Systems*. 390–399.

[30] Jiakai Tang, Sunhao Dai, Teng Shi, Jun Xu, Xu Chen, Wen Chen, Jian Wu, and Yuning Jiang. 2025. Think before recommend: Unleashing the latent reasoning power for sequential recommendation. *arXiv preprint arXiv:2503.22675* (2025).

[31] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable item tokenization for generative recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 2400–2409.

[32] Ye Wang, Jiahao Xun, Minjie Hong, Jieming Zhu, Tao Jin, Wang Lin, Haoyuan Li, Linjun Li, Yan Xia, Zhou Zhao, et al. 2024. Eager: Two-stream generative recommender with behavior-semantic collaboration. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3245–3254.

[33] Zhipeng Wei, Kuo Cai, Junda She, Jie Chen, Minghao Chen, Yang Zeng, Qiang Luo, Wencong Zeng, Ruiming Tang, Kun Gai, et al. 2025. OneLoc: Geo-Aware Generative Recommender Systems for Local Life Service. *arXiv preprint arXiv:2508.14646* (2025).

[34] Yi Xu, Moyu Zhang, Chaofan Fan, Jinxin Hu, Xiaochen Li, Yu Zhang, Xiaoyi Zeng, and Jing Zhang. 2025. MMQ-v2: Align, Denoise, and Amplify: Adaptive Behavior Mining for Semantic IDs Learning in Recommendation. *arXiv preprint arXiv:2510.25622* (2025).

[35] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[36] Yuhao Yang, Zhi Ji, Zhaopeng Li, Yi Li, Zhonglin Mo, Yue Ding, Kai Chen, Zijian Zhang, Jie Li, Shuanglong Li, et al. 2025. Sparse meets dense: Unified generative recommendations with cascaded sparse-dense representations. *arXiv preprint arXiv:2503.02453* (2025).

[37] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).

[38] Jun Zhang, Yi Li, Yue Liu, Changping Wang, Yuan Wang, Yuling Xiong, Xun Liu, Haiyang Wu, Qian Li, Enming Zhang, et al. 2025. GPR: Towards a Generative Pre-trained One-Model Paradigm for Large-Scale Advertising Recommendation. *arXiv preprint arXiv:2511.10138* (2025).

[39] Zhaoqi Zhang, Haolei Pei, Jun Guo, Tianyu Wang, Yufei Feng, Hui Sun, Shaowei Liu, and Aixin Sun. 2025. OneTrans: Unified Feature Interaction and Sequence Modeling with One Transformer in Industrial Recommender. *arXiv preprint arXiv:2510.26104* (2025).

[40] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1435–1448.
[41] Zuowu Zheng, Ze Wang, Fan Yang, Jiangke Fan, Teng Zhang, Yongkang Wang, and Xingxing Wang. 2025. EGA-V2: An End-to-end Generative Framework for Industrial Advertising. arXiv:2505.17549 [cs.IR] https://arxiv.org/abs/2505.17549
[42] Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, et al. 2025. Onerec-v2 technical report. *arXiv preprint arXiv:2508.20900* (2025).
[43] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.
[44] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, et al. 2025. Rankmixer: Scaling up ranking models in industrial recommenders. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6309–6316.

A Related Work

A.1 Live-streaming Recommendation

Live-streaming is an emerging popular media that allows for direct interaction between users and authors, which has been widely explored on many online platforms, such as Kuaishou [1] and TikTok [17]. Compared with short videos or products, the most significant difference of live-streaming lies in the dynamic changes of its content. Therefore, live-streaming recommendation have strict requirements for real-time content understanding and fast response. Early methods such as LiveRec [29] only focus on historical interactions and fail to incorporate content information. ContentCTR [5] and MMBee [7] introduce multimodal information about the granularity of the entire live-streaming and develop a more comprehensive understanding, but such rough aggregated information may not match the real-time state.

In order to achieve dynamic perception of live-streaming, Moment [1] changes the data-streaming engine to a real-time segment report manner. On this basis, either FARM [18] introduces cross-domain knowledge or LARM [21] adds LLM encoding, both of which fully enhance the ability of modeling relationship between users and authors. However, these discriminative methods still rely on pointwise scoring and ranking, and are limited by the pre-filtering of the candidate set, making end-to-end prediction at scale challenging. LiveForesighter [3, 22] generates predictions for the next product in online-shopping live-streaming, but the results are still incorporated into a feature cross-fusion module, without changing the essence of the recommendation paradigm.

Therefore, shifting from a cascaded discriminative architecture to an end-to-end generative paradigm is crucial to meet the demand for immediate feedback in live-streaming recommendation.

A.2 Generative Recommendation

Generative models, especially LLMs [9, 35], have achieved breakthrough progress in various fields and demonstrated great potential in recommendation systems. The main research line of LLM-based generative recommendation is to generate personalized suggestions through prompts or fine-tuning using pretrained LLMs [14]. These methods either leverage contextual awareness [13, 40] or stimulate reasoning capability [8, 30] of LLMs to align recommendation tasks, but often require additional data restructuring or time consumption.

Another insight from LLMs lies in the benefits of scaling, which has been proven the effectiveness within the cascaded stages [37, 44]. To achieve scaling in recommendation and overcome the computational fragmentation caused by cascaded architectures, directly training a real end-to-end large recommendation model has attracted attentions from the industry. TIGER [28] establishes the foundational architecture for this paradigm, which represents items as sequence of discrete tokens from codebooks learned by RQ-VAE [16], then uses encoder-decoder model to directly predict the Semantic ID of the next item. OneRec [6] implements this end-to-end architecture in industry, replacing the cascaded pipeline, and introduces preference alignment to optimize generation results. On this basis, OneLoc [33] and OneSug [10] adapt this idea in the scenarios of local life and query suggestion. COBRA [36] unifies the generation of sparse and dense, EGA-V2 [41] integrates multiple tasks, and MMQ [34] aligns content and behavior for Semantic ID learning. These methods have significantly advanced this direction, yet remain underexplored in live-streaming scenario.

B Experiment Details

B.1 Experimental Settings

B.1.1 ***Dataset.*** We construct training and testing datasets based on large-scale industrial live-streaming platform logs from the Kuaishou App. We collect data for nearly two months, including 400 million users and 3 million authors, as well as billions of interaction data between them, covering long-view and click records. The data from the last day are used for testing, and the data from the remaining dates are used for training. All subsequent offline and ablation experiments are conducted on this dataset.

B.1.2 ***Evaluation Metrics.*** Following most prior works, the evaluation considers both recall and ranking performance, therefore we adopt Accuracy (ACC), Hit Rate (HR) and Mean Reciprocal Ranking (MRR) as our offline evaluation metrics. We also utilize Reward evaluated by the deployed online ranking model to quantify its recognition of the generative results. For online A/B tests, we use core market indicators to demonstrate the effectiveness to online business. Detailed metric definitions are provide in Appendix C.2.

B.1.3 ***Baselines.*** We compare OneLive with competitive baselines within two groups of works: (i) **Traditional Models**: classic sequential recommendation method **SASRec** [15], adaptive long sequence compression method **KuaiFormer** [20] and heterogeneous structure-aware graph method **GNN** [11] variant; (ii) **Generative Models**: original SID-based generative method **TIGER** [28] and successful solution in industrial scenario **OneRec** [6].

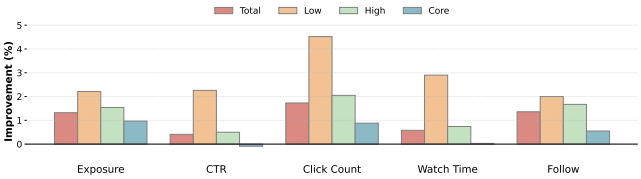
B.2 Overall Experiemnt Analysis

Notably, not all generative methods consistently outperform traditional methods. In particular, TIGER shows a significant performance drop, which contradicts with prior industrial reports [6, 33]. We attribute this to the fact that in the dynamic scenario of live-streaming, TIGER models the SID sequence corresponding to the user's historical interaction items (MLLM Codes of authors in reproduction experiment, while even worse with SID Codes for capturing instant rather than synthesized content) for subsequent prediction, which hinders the model's ability to distribute dynamically from such coarse-gained and static representations. This highlights and

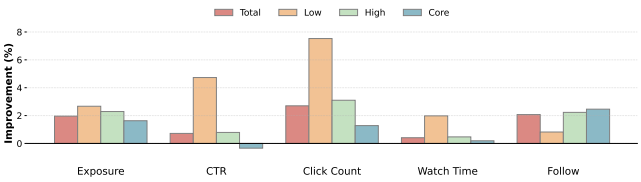
(a) Talent Show Author.

(b) E-Commerce Author.

Figure 6: Cases of SID updates with live-streaming evolving.



(a) Kuaishou



(b) Kuaishou Lite

Figure 5: Stratified test of online performance.

indirectly confirms the core challenge of providing real-time recommendation with evolving content in live-streaming scenario.

However, OneLive and OneRec demonstrate strong adaptability in industrial settings. Beyond the discrete feature SID, the incorporation of extensive user and author side information significantly enhances the model's generalization capability in complex real-world environments. Moreover, the introduction of temporal modeling and enriched training objectives better aligns with the requirements of live-streaming recommendation.

B.3 Online Stratified Test

We further conduct a stratified test to evaluate the generalization over distinct crowd groups, including low-active users, high-active users and core-paid users. The performance across these groups are visualized in Figure 5. Except for minor decrease in a few indicators, the results show that our unified generative model achieves consistent and significant improvements across all groups. Notably, low-active users exhibit the largest gains, which might be attributed to the long-tail neglect problem in conventional cascade pipelines. In contrast, OneLive demonstrates an astonishing potential to meet diverse preferences across active levels.

B.4 Case Study

We demonstrate several cases in Figure 6. We can observe that, our live-streaming SID generated from dynamic tokenizer can capture the current live content of authors and update in real time, ensuring continuous continuity with live-streaming evolving.

C Metrics Definitions

C.1 Codebook Analysis

We use **Utilization Rate (UR)** and **Collision Rate (CR)** to evaluate the quality of SID code. Specifically:

- **Utilization Rate (UR)**: It measures the ratio of the number of distinct codes appearing at each layer relative to the size of the entire codebook size. For a given layer L_i, let C_{total} represent the total size of the codebook, and C_{used} denote the number of distinct codes that have appeared at this layer. The $\mathbf{UR}_{L_i}$ can be expressed as:

$$\mathbf{UR}_{L_i} = \frac{C_{used}}{C_{total}}. \tag{30}$$

- *Collision Rate (CR)*: This measures the degree of overlap from different perspectives (*e.g.*, SID or Author).
  Specifically, in the SID dimension, a SID may be associated with multiple different authors. Therefore, the $\mathbf{CR}_{SID}$ measures the ratio of the number of SIDs $C_{collision\_sid}$ that are associated with multiple authors relative to the total number of distinct SIDs C_{sid}:

$$\mathbf{CR}_{SID} = \frac{C_{collision\_sid}}{C_{sid}}. \tag{31}$$

  In the Author dimension, multiple authors may share the same SID. Therefore, the $\mathbf{CR}_{Author}$ measures the ratio of the number of authors $C_{collision\_author}$ who share same SID with others relative to the total number of distinct authors C_{author}:

$$\mathbf{CR}_{Author} = \frac{C_{collision\_author}}{C_{author}}. \tag{32}$$

C.2 Experiment Comparison

We use **Accuracy (ACC)** as the metric for training, and **Hit Rate (HR)**, **Mean Reciprocal Ranking (MRR)** and **Reward** for inference. Specifically:

- **Accuracy (ACC)**: This measures the average accuracy of multi-level SID hits during training. Formally:

$$\text{ACC@}all = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(\hat{q}_i = q_i), \tag{33}$$

where N is the total SID in training samples, $\hat{q}_i$ is the predicted SID and q_i is the true SID.

- **Hit Rate (HR)**: This measures the correct answer appears in the Top-k predictions when performing inference with beam search. Formally:

$$\mathbf{HR}@k = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(y_i \in \text{Top-k}), \tag{34}$$

where y_i is the ground truth and Top-k refers to the predictions generated by the model.

- **Mean Reciprocal Ranking (MRR)**: This measures the rank of the correct label within the Top-k predictions. Formally:

$$\mathbf{MRR}@k = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\text{rank}_i}, \tag{35}$$

where rank_i is the position of the correct label in the Top-k predictions.

- **Reward**: This measures the recognization of online system based on how well it ranks the predictions. It obtains the XTR scores by transferring the user and author pairs obtained from the prediction results into the ranking model. Formally:

$$\mathbf{Reward}_{XTR} = \frac{1}{N}\sum_{i=1}^{N}\text{Ranking}_{XTR}(\text{User}, y_i). \tag{36}$$